

گزارش پروژه سوم نرم افزار ریاضی

درونیابی لاگرانژ و اسپلاین

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline
from sympy import symbols,simplify

x = np.array([1,2,3,4,5,6])
y = np.array([1,3,5,8,5,2])
n = len(x)
X = symbols('X')
P = 0
for i in range(n):
    L = 1
    for j in range(n):
        if j != i:
            L = L * (X-x[j]) / (x[i]-x[j])
    P = P + y[i] * L

P_simplified = simplify(P)
print("\n")
print("lagrange interpolation polynomial (degree {}):".format(n-1))
print("\n")
print(P_simplified)
print("\ncoefficients :")
poly_coeffs = [float(P_simplified.coeff(X, k)) for k in range(n)]
for k, coeff in enumerate(poly_coeffs):
    print(f" x^{k}: {coeff:.6f}")
print("\n")

def lagrange(x_val):
    result = 0
    for i in range(n):
        L = 1
        for j in range(n):
            if j != i:
                L = L * (x_val-x[j]) / (x[i]-x[j])
        result = result + y[i] * L
    return result

cs = CubicSpline(x,y,bc_type='natural')
```

```

print("\n")
print("Cubic Spline Coefficients:")
print("[x_i, x_{i+1}]:")
print("S_i(x) = a_i + b_i*(x-x_i) + c_i*(x-x_i)^2 + d_i*(x-x_i)^3")
print("\n")

for i in range(n-1):
    coeffs = cs.c[:,i]
    a_i = coeffs[3]
    b_i = coeffs[2]
    c_i = coeffs[1]
    d_i = coeffs[0]
    print(f"Interval {i+1}: [{x[i]:.1f}, {x[i+1]:.1f}]")
    print(f"  a_{i+1} = {a_i:.6f}")
    print(f"  b_{i+1} = {b_i:.6f}")
    print(f"  c_{i+1} = {c_i:.6f}")
    print(f"  d_{i+1} = {d_i:.6f}")

xx = np.linspace(min(x),max(x),200)
yy_lagrange = np.array([lagrange(xi) for xi in xx])
yy_spline = cs(xx)
plt.figure(figsize=(12,8))
plt.plot(x,y,'ro',markersize=10,label='main points')
plt.plot(xx,yy_lagrange,color="blue",label='lagrange')
plt.plot(xx,yy_spline,'--',label='cubic spline')
plt.xlabel('x',fontsize=12)
plt.ylabel('y',fontsize=12)
plt.title('lagrange vs cubic spline',fontsize=14)
plt.legend(fontsize=11)
plt.grid(True,alpha=0.3)
plt.tight_layout()
plt.show()

print("\n")
print("comparison of values at midpoints:")
print("\n")
test = np.array([1.5,2.5,3.5,4.5,5.5])
print(f"{'x':>8}{'lagrange':>12}{'spline':>12}{'difference':>12}")
print("=====")
for i in test:
    yLAG = lagrange(i)
    ySP = cs(i)
    diff = abs(yLAG - ySP)
    print(f"{i:>8.2f}{yLAG:>12.6f}{ySP:>12.6f}{diff:>12.6f}")

```

در این کد، ابتدا مختصات نقاط داده در دو آرایه X و Y ذخیره می‌شوند و تعداد نقاط نیز در متغیر n قرار می‌گیرد. سپس با استفاده از فرمول درون‌یابی لاگرانژ، چندجمله‌ای درون‌یاب ساخته می‌شود. برای این منظور، در یک حلقه، چندجمله‌ای پایه لاگرانژ برای هر نقطه محاسبه شده و در مقدار متناظر آن ضرب می‌شود. مجموع این چندجمله‌ای‌ها، چندجمله‌ای نهایی لاگرانژ را تشکیل می‌دهد. در ادامه، این چندجمله‌ای با استفاده از تابع `simplify` ساده شده و به همراه ضرایب آن در خروجی نمایش داده می‌شود. پس از آن، تابعی برای محاسبه مقدار درون‌یابی لاگرانژ در هر نقطه دلخواه تعریف می‌شود تا بتوان مقدار تابع را در نقاط مختلف محاسبه کرد.

در مرحله بعد، اسپلاین مکعبی طبیعی ایجاد می‌شود. سپس ضرایب مربوط به هر بازه شامل ضرایب جمله ثابت، خطی، درجه دوم و درجه سوم استخراج شده و برای تمامی بازه‌ها در خروجی نمایش داده می‌شوند. این ضرایب معادله اسپلاین را در هر بازه مشخص می‌کنند.

در ادامه، مجموعه‌ای از نقاط با فاصله‌های بسیار کوچک در بازه داده‌ها تولید می‌شود تا منحنی‌های حاصل از هر دو روش با دقت مناسب رسم شوند. سپس مقدار تابع در این نقاط با استفاده از روش لاگرانژ و اسپلاین مکعبی محاسبه شده و هر دو منحنی به همراه نقاط اصلی داده روی یک نمودار رسم می‌شوند. این نمودار امکان مقایسه بصری عملکرد دو روش را فراهم می‌کند.

در بخش پایانی، برای مقایسه عددی دو روش، چند نقطه میانی بین داده‌های اصلی انتخاب می‌شود. مقدار تابع در این نقاط با استفاده از هر دو روش محاسبه شده و اختلاف قدرمطلق نتایج نیز نمایش داده می‌شود. این مقایسه نشان می‌دهد که در نقاط میانی، دو روش تا چه اندازه نتایج مشابه یا متفاوتی تولید می‌کنند.

خروجی این کد به شرح زیر می‌باشد:

```
lagrange interpolation polynomial (degree 5):
```

```
7*X**5/40 - 71*X**4/24 + 147*X**3/8 - 1249*X**2/24 + 1369*X/20 - 31
```

```
coefficients :
```

```
x^0: -31.000000
```

```
x^1: 68.450000
```

```
x^2: -52.041667
```

```
x^3: 18.375000
```

```
x^4: -2.958333
```

```
x^5: 0.175000
```

Cubic Spline Coefficients:

$[x_i, x_{i+1}]$:

$$S_i(x) = a_i + b_i(x-x_i) + c_i(x-x_i)^2 + d_i(x-x_i)^3$$

Interval 1: [1.0, 2.0]

$$a_1 = 1.000000$$

$$b_1 = 2.186603$$

$$c_1 = 0.000000$$

$$d_1 = -0.186603$$

Interval 2: [2.0, 3.0]

$$a_2 = 3.000000$$

$$b_2 = 1.626794$$

$$c_2 = -0.559809$$

$$d_2 = 0.933014$$

Interval 3: [3.0, 4.0]

$$a_3 = 5.000000$$

$$b_3 = 3.306220$$

$$c_3 = 2.239234$$

$$d_3 = -2.545455$$

Interval 4: [4.0, 5.0]

$$a_4 = 8.000000$$

$$b_4 = 0.148325$$

$$c_4 = -5.397129$$

$$d_4 = 2.248804$$

Interval 5: [5.0, 6.0]

$$a_5 = 5.000000$$

$$b_5 = -3.899522$$

$$c_5 = 1.349282$$

$$d_5 = -0.449761$$

comparison of values at midpoints:

x	lagrange	spline	difference
1.50	2.949219	2.069976	0.879243
2.50	3.503906	3.790072	0.286166
3.50	6.871094	6.894737	0.023643
4.50	7.425781	7.005981	0.419800
5.50	2.042969	3.331340	1.288371

با بررسی نمودار مشاهده می‌شود که هر دو روش از تمامی نقاط داده عبور می‌کنند، اما رفتار آن‌ها بین نقاط متفاوت است. در روش لاگرانژ، یک چندجمله‌ای واحد با درجه بالا برای کل بازه ساخته می‌شود؛ به همین دلیل ممکن است در برخی قسمت‌ها نوسانات بیشتری ایجاد شود، به‌ویژه زمانی که تعداد نقاط افزایش یابد. در مقابل، روش اسپلاین مکعبی بازه را به چند بخش تقسیم کرده و در هر بخش یک چندجمله‌ای درجه سوم مجزا ایجاد می‌کند. به همین دلیل، منحنی حاصل معمولاً هموارتر بوده و تغییرات آن طبیعی‌تر است. در روش لاگرانژ چند جمله ای زیر را داریم:

$$P(x) = \frac{7}{40}x^5 - \frac{71}{24}x^4 + \frac{147}{8}x^3 - \frac{1249}{24}x^2 + \frac{1369}{20}x - 31$$

با توجه به جدول اختلاف‌ها مشخص می‌شود که اختلاف این دو روش در مرکز بازه کم است اما در دو سر باز این اختلاف به حداکثر می‌رسد. همچنین مشخص می‌شود که روش اسپلاین رفتار و تغییرات ملایم‌تری دارد.

خواندن داده از اکسل و نمایش نمودارها

```
import pandas as pd
import matplotlib.pyplot as plt

file = "پروژه ۳.xlsx"
excel = pd.read_excel(file)
size = excel.iloc[:,0]
alg1 = excel.iloc[:,1]
alg2 = excel.iloc[:,2]
alg3 = excel.iloc[:,3]

plt.figure(figsize=(12,8))
width = 0.25
x = range(len(size))
plt.bar([i-width for i in x],alg1,width,label="algorithm 1")
plt.bar(x,alg2,width,label="algorithm 2")
plt.bar([i+width for i in x],alg3,width,label="algorithm 3")
plt.xticks(x,size)
plt.xlabel("Size")
plt.ylabel("Time")
plt.title("Bar Chart")
plt.legend()
plt.show()

plt.figure(figsize=(12,8))
plt.plot(size,alg1,marker="o",label="algorithm 1")
```

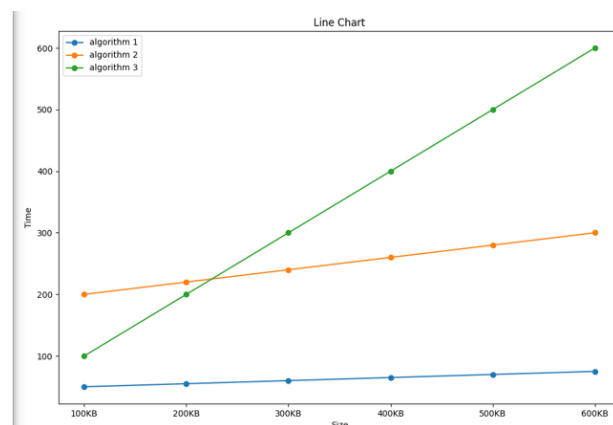
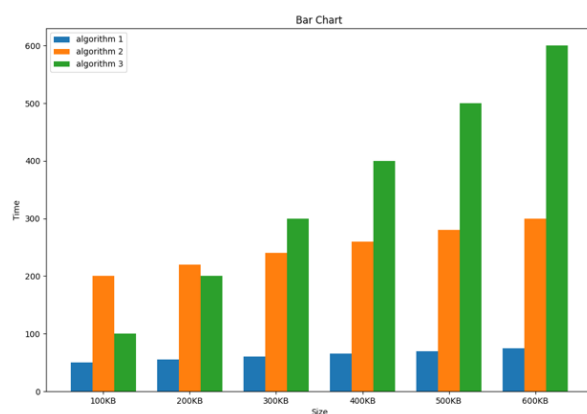
```
plt.plot(size,alg2,marker="o",label="algorithm 2")
plt.plot(size,alg3,marker="o",label="algorithm 3")
plt.xlabel("Size")
plt.ylabel("Time")
plt.title("Line Chart")
plt.legend()
plt.show()

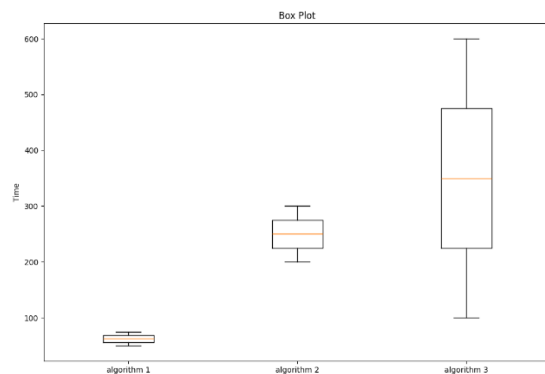
plt.figure(figsize=(12,8))
plt.boxplot([alg1,alg2,alg3],labels=["algorithm 1","algorithm 2","algorithm 3"])
plt.title("Box Plot")
plt.ylabel("Time")
plt.show()

print("Average Algorithm 2 =",alg2.mean())
```

در این بخش ابتدا نیازمند این هستیم که کتابخانه‌های مورد نیاز را فراخوانی کنیم. بعد از فراخوانی در مرحله اول، فایل اکسل از مسیر مشخص شده خوانده می‌شود. سپس اطلاعات موجود در هر ستون، شامل اندازه داده‌ها، زمان اجرای الگوریتم اول، الگوریتم دوم و الگوریتم سوم، به صورت خودکار به کمک خط ۵ تا ۹ کد از فایل اکسل استخراج و در چهار متغیرهای جداگانه ذخیره می‌شوند.

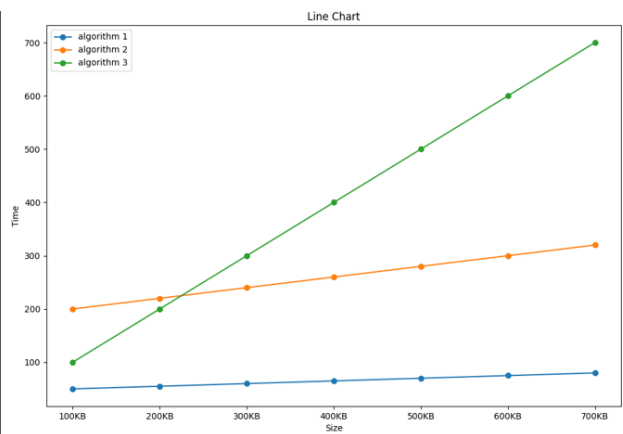
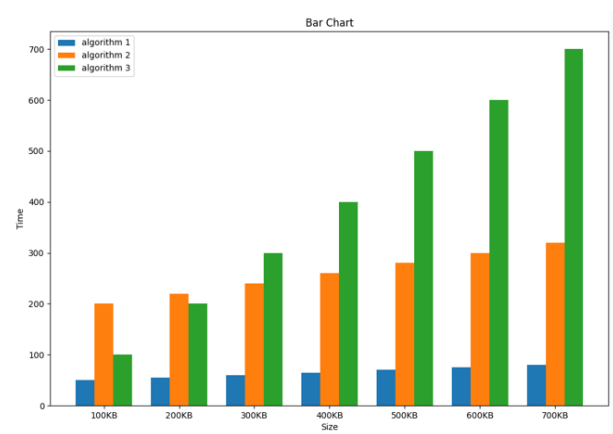
در ادامه، به ترتیب سه نوع نمودار شامل نمودار میله‌ای، نمودار خطی و نمودار جعبه‌ای رسم می‌شوند. در کد برنامه برای هر نمودار جزئیات مورد نیاز اضافه شده است. نمودار میله‌ای امکان مقایسه مستقیم زمان اجرای سه الگوریتم را در اندازه‌های مختلف داده فراهم می‌کند. نمودار خطی روند تغییرات زمان اجرای هر الگوریتم را با افزایش حجم داده‌ها نمایش می‌دهد و نمودار جعبه‌ای نیز نحوه پراکندگی داده‌ها، میانه، دامنه تغییرات و نقاط پرت احتمالی را مشخص می‌کند. برای ۶ داده و قبل از اضافه کردن داده جدید خروجی‌های کد به شرح زیر می‌باشند:

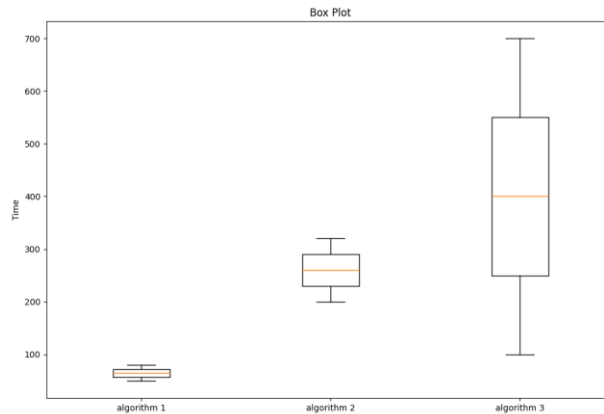




با توجه به این سه نمودار مشخص است که در الگوریتم اول و دوم با افزایش حجم داده‌ها تغییر قابل توجهی در زمان اجرا به وجود نمی‌آید اما در الگوریتم سوم میزان افزایش زمان بسیار بیشتر است. به گونه‌ای که در حجم اول زمان اجرای این الگوریتم از الگوریتم دوم هم کمتر است اما با افزایش شش برابری حجم، زمان اجرای آن به دوبرابری زمان اجرای الگوریتم دوم می‌رسد. همچنین مشخص می‌شود که میزان زمان اجرای هر سه الگوریتم با افزایش حجم داده به صورت خطی رشد می‌کند. از نمودار جعبه نیز نتیجه می‌شود که الگوریتم اول کمترین میانه را دارد. میانه کمتر نشان دهنده زمان اجرای کمتر می‌باشد. همچنین به ترتیب الگوریتم‌های اول، دوم و سوم دارای کمترین فاصله میان چارکی هستند. هرچه این فاصله کمتر باشد، نشان دهنده این است که تغییرات زمان اجرا از تغییر حجم شدت کمتری می‌پذیرد.

پس از افزودن داده جدید نتایج و نمودارهای زیر حاصل می‌شوند.





نتایج این بخش نتایج بخش قبل را تایید می کنند. الگوریتم سوم با شیب تند تری نسبت به دو الگوریتم دیگر رشد می کند. اطلاعات بخش باکس پلات نیز همانند بخش قبل هستند و تغییر زیادی نکرده اند.

در بخش بعد، میانگین زمان اجرای الگوریتم دوم با استفاده از داده های موجود در ستون مربوطه محاسبه می شود. این مقدار با استفاده از تابع میانگین به دست آمده و روی مقدار ستون الگوریتم ۲ که در خط ۸ کد از فایل اکسل استخراج شده بود اعمال می شود و در خروجی برنامه نمایش داده می شود. این خروجی به صورت زیر می باشد.

Average Algorithm 2 = 250.0

محاسبه تابع با روش نوزنقه ای و سیمپسون و انتگرال گاوسی

پیاده سازی این بخش در زبان برنامه نویسی پایتون انجام شده است.

```
import numpy as np

exact = np.exp(1)-1
head = 0
tail = 1
n = 10
h = (tail - head) / n
x = np.linspace(head,tail,n+1)
y = np.exp(x)
trap = h * ((y[0]+y[n]) / 2+np.sum(y[1:n]))
simpson = h/3 * (y[0]+ y[n]+ 4*np.sum(y[1:n:2])+ 2*np.sum(y[2:n-1:2]))
t = np.array([-1/np.sqrt(3),1/np.sqrt(3)])
w = np.array([1,1])
xg = (tail-head) / 2*t + (head+tail)/2
```



```

gauss = (tail-head) / 2*np.sum(w*np.exp(xg))
print("exactValue   =",exact)
print("zozangei     =",trap)
print("Simpson       =",simpson)
print("Gaussian      =",gauss)
print("\n")
print("zozangeError  =",abs(exact-trap))
print("SimpsonError  =",abs(exact-Simpson))
print("GaussianError=",abs(exact-gauss))

```

در این کد ابتدا مقدار دقیق انتگرال، یعنی $(e-1)$ ، که پاسخ تحلیلی مسئله است، با استفاده از کتابخانه `numpy` محاسبه می‌شود. سپس ابتدا و انتهای بازه انتگرال گیری مشخص شده و به ترتیب در متغیرهای مربوطه ذخیره می‌شوند. در مرحله بعد، بازه انتگرال گیری به ۱۰ قسمت مساوی تقسیم می‌شود و طول هر زیربازه محاسبه شده و در متغیر `h` قرار می‌گیرد تا در روش‌های مختلف مورد استفاده قرار گیرد. همچنین تمامی نقاط بازه با استفاده از تابع `linspace` تولید شده و در متغیر `X` ذخیره می‌شوند. در این مسئله، نقاط بازه از صفر تا یک با فاصله یک دهم تولید می‌شوند. در مرحله بعدی، مقدار تابع $f(x)$ در تمامی نقاط موجود در `X` محاسبه می‌شود. سپس فرمول روش دوزنقه‌ای که به صورت زیر است، در برنامه پیاده‌سازی می‌شود:

$$I \approx h \left(\frac{f(x_0) + f(x_n)}{2} + \sum_{i=1}^{n-1} f(x_i) \right)$$

در مرحله بعدی فرمول روش سیمپسون و روش گاوسی را در پایتون پیاده‌سازی می‌کنیم. برای روش سیمپسون جمع اولین نقطه و آخر نقطه و چهار برابر مقدار اندیس‌های فرد و دو برابر مقدار اندیس‌های زوج را حساب کرده و در $h/3$ ضرب می‌کنیم. برای انتگرال گاوسی ابتدا دو نقطه استاندارد انتگرال گاوسی را تعریف می‌کنیم و در `t` قرارشان می‌دهیم. در مرحله بعدی وزن هر نقطه محاسبه می‌شود و بازه انتگرال گاوسی نیز اصلاح می‌شود. در نهایت نیز این مقادیر در فرمول اصلی گاوسی قرار می‌گیرد.

پس از انجام این کارها خروجی کد که شامل مقدار دقیق و مقادیر حاصل از سه روش مختلف و خطای آن‌ها می‌باشد در خروجی پرینت می‌شوند خروجی‌های این کد در قالب کد به صورت زیر می‌باشند:

```

exactValue   = 1.718281828459045
zozangei     = 1.7197134913893146

```

```
Simpson      = 1.7182827819248236
Gaussian     = 1.717896378007504

zozangeError = 0.0014316629302695283
SimpsonError = 9.53465778552598e-07
GaussianError= 0.0003854504515410362
```

با بررسی نتایج به این نتیجه می‌رسیم که روش سیمپسون بهترین عملکرد را در بین روش‌های استفاده شده داشته است و روش دوزنقه علی‌رغم خطای کم، نسبت با دیگر روش‌ها خطای بیشتری داشته است که علت این مشکل حالت خاص تابع است که دارای شیب تندی می‌باشد و در نتیجه در دوزنقه‌ها با خطای زیادی مواجه می‌شوند.